

AgentIA — First Measured Baseline

The first real generation run: what it cost, how it failed, and what it means for the optimization approach

AgentIA Engineering

2026-09-29

What this document records. On 2026-09-29 the platform executed its first genuine end-to-end generation session. Before it, no session had ever produced a measured outcome: the verification metrics, the cost store and the optimization records were all empty. This is the first data, and the first honest answer to *how reliable is this agent*. It also corrects a category error about which gate is which, because that error would misdirect the optimization design.

CORRECTION, 2026-09-29, after re-measurement. The central finding of this document is withdrawn.

It reported *systematic procedural non-compliance*: every stage spending its entire correction budget, “not one passed on the first attempt”, and concluded that this was precisely the signal a reflective skill optimiser needs. That observation was an artefact of a **response-parsing bug**, not of model behaviour.

- The extractor accepted only a bare JSON object, or code fences carrying the artifact path on the fence line. The request never stated a wire format. The shapes a real model actually emits — fenced JSON, prose-wrapped JSON, and code blocks whose path sits in a heading — failed to parse, so the attempt was consumed and the stage exhausted *without ever producing a candidate for the gate to judge*.
- A second defect recorded *no verdict at all* for a stage that passed on its first attempt, so **passed = None** covered two opposite realities: “nothing was judged” and “judged and accepted”. That is why the record read as though nothing had been wrong.

Re-measured after fixing both — five blueprints, one session each: **25 of 25 stage runs succeeded on their FIRST request**, with zero corrections; 83 artifacts persisted; **conformance 100 for every session, density 0.0, and zero findings in every final artifact set**.

Consequences. (i) There is no procedural non-compliance to correct: the agent produced compliant output first time, on every stage, for every blueprint. (ii) The conformance measure does not vary, so the optimiser’s own gate — *the measure must vary before anything is built to exploit it* — **fails**, and the optimiser is for that reason not built. (iii) The reading in §4.1 and the critical path named in §4.2 are withdrawn.

What survives, and it is why this document is kept rather than deleted. The diagnostic channel did what it was built to do: running the real system found three defects that a suite of 500+ passing tests could not — the parser above, a “verified” session whose build never ran, and a diagnostic record silently overwritten by test data. **The instrument was worth building. The loop it was built to justify has nothing to optimise.**

Contents

1	The result	2
2	How it failed	2

3	The gates are not the same gate	3
4	What this means for the reflective approach	3
4.1	The result is encouraging for the approach	4
4.2	But the evidence is currently lost	4
5	The proposed schema	4
6	Next steps, in dependency order	5
7	Telemetry: is this in MLflow?	5
8	Defects found by running	6
9	Honest position	6

1 The result

One session, held-out blueprint `minimal`, provider `deepseek`, model `deepseek-flash`.

Measure	Value
Model calls	15
Input tokens	26,351
Output tokens	139,679
Cost	\$0.085295889
Calls priced	15 of 15
Terminal state	BLOCKED
Artifacts persisted	0

The cost figure is the first the project has ever produced. Feature 013’s cost tracing works: every call was recorded, priced against the published rate table, and reconciled. The telemetry, however, did *not* reach MLflow — see §7.

2 How it failed

Every stage is allowed one initial request plus two corrections. The observed distribution is the finding:

Stage	Calls	Corrections	Outcome	Note
SCAFFOLDER	3	2	passed	43.4s and 11,045 output tokens on the rejected fir
DOMAIN	3	2	passed	
SERVICE	3	2	passed	
CONTROLLER	3	2	passed	
TEST	3	2	EXHAUSTED	<i>never converged</i>

Every stage consumed its entire correction budget. Not one passed on the first attempt. This is a systematic, procedural failure pattern, not random noise — and it is the single most useful thing the run produced.

WITHDRAWN. This claim is false. The stages exhausted because their responses could not be parsed, not because they were rejected for non-compliance. Re-measured after the parser was

fixed: **25 of 25 stage runs across five blueprints succeeded on their first request**, with no corrections at all. There is no “systematic, procedural failure pattern”; there was a transport bug. See the correction notice at the front of this document.

The terminal error was:

```
[TEST] correction budget exhausted after 2 correction attempts;
human intervention required
```

That is constitution Principle V working as specified: bounded autonomy, then human intervention. The session did not silently succeed, and it did not loop.

3 The gates are not the same gate

A correction that matters for the design. There are **two unrelated mechanisms**, both called “the gate”:

	Compliance gate	Optimization gate
Origin	Feature 011	Feature 014 (SkillOpt)
Lives in	<code>stages/runner.py</code>	<code>scripts/skillopt/gate.py</code>
Judges	one model <i>response</i>	one <i>skill document</i>
On rejection	forces a retry, bounded at 3	keeps the current skill
Scope	within a session	across sessions
Status	active — this is what fired	deferred, never run

The gate that rejected fifteen responses is not a SkillOpt-style gate. It is the generation-time compliance validator. The SkillOpt gate compares two skill documents by executing fresh sessions and accepting only a strict improvement; it has never run.

The distinction is not pedantic. The compliance gate rejects *within* a session and its verdicts are discarded once the session ends. The optimization gate rejects *across* sessions and needs its evidence to persist. Confusing them leads to the belief that the platform already has an optimization loop — it does not; it has a validator.

4 What this means for the reflective approach

The intended approach is CoEvoSkills-style: a reflective language model proposes skill edits, guided by structured failure diagnostics, admitted only on measured evidence. CoEvoSkills reported **+40.5 points** on SkillsBench with that shape, and its own ablation is the reason the shape matters:

CoEvoSkills variant	Score
Full (co-evolving diagnostic verifier)	71.1
Opaque pass/fail oracle only	41.1 (−30.0)
No evolution at all	42.4 (−28.7)

A loop fed only a pass/fail verdict performs no better than no loop.


```

targeting the RECURRING rules
|
v
candidate skill -> fresh sessions -> measure
|
v
admit only on measured contribution;
evict skills below the evidence floor

```

Mapping onto the existing code:

Component	Where	State
Surrogate verifier (the 30-point component)	services/conformance_diagnostics.py	built
Per-stage attribution	same, reading the stage journal	built
Evidence persistence	models/diagnostics.py	built, broken on the blocked path
Corpus report	services/corpus_report.py	built
Skill generator (reflection)	scripts/skillopt/reflect.py	exists, fed the wrong signal
Evidence gate (leave-one-out)	services/skill_contribution.py	not built
Eviction with a floor	scripts/skillopt/currency.py	not built

6 Next steps, in dependency order

1. **Capture artifacts on the blocked path.** Without it the diagnostic channel is empty exactly when it matters most, and every step below is unevaluable. *This is the critical path.*
2. **Feed the reflections the diagnostics.** The reflector currently receives a derived build exit code, a terminal status and artifact paths. It should receive the rule histogram: which rules fired, how often, at which stage. The channel exists; the wiring does not.
3. **Measure per-skill contribution** by leave-one-out, so an edit can be attributed to a rule rather than to a document.
4. **Add eviction with an evidence floor.** A library that can only grow degrades until injecting a skill is worse than injecting nothing.
5. **Grow the task corpus.** Five blueprints cannot support inference: one task moves the pass rate 25 points, and the arithmetic floor for any paired comparison at that size cannot reach $p < 0.05$ under any outcome.
6. **Only then** run the optimization loop, scored on conformance rather than on build success.

7 Telemetry: is this in MLflow?

No. The mirror is wired — `cost/mlflow_sink.py` exposes `mirror_call_record`, and `cost/recording.py` calls it on every recorded call — but MLflow is **not installed** in this environment, so the lazy `import mlflow` fails and each mirror attempt returns `False`.

Destination	Status	Role
backend/cost_tracking.db	populated	system of record
MLFLOW_TRACKING_URI	localhost:5000, unreachable	best-effort mirror

This is deliberate: feature 013 made the local store authoritative and the mirror a non-event when absent, so the cost report is deterministic and works offline. The figures in this document come from the local store. To see them in MLflow, install `mlflow` and run a tracking server; no code change is required.

8 Defects found by running

None of the following were reachable from the test suite. All six were found by executing the real thing, which is the argument for executing it.

#	Defect	Why the suite missed it
1	MODEL mode could not run through the graph at all. LangGraph filters state to the declared schema, and <code>llm_api_key</code> was not declared, so it was silently stripped between entry and the first node	the 011 tests call <code>run_stage</code> with a hand-built state and bypass the graph
2	A table created before a column existed crashed the report: <code>create_all</code> creates missing <i>tables</i> , never missing <i>columns</i>	the test database is rebuilt fresh every run, so no test saw an older table
3	A crashed session was not recorded, so a batch of four reported three	tests used well-behaved runners
4	An empty artifact set rendered as <i>verified clean</i>	caught by the FR-004 test once written, not before
5	The sandbox volume mount was unreadable on this host (needs <code>:Z</code> under labelled rootless podman), so Maven found no pom and every session blocked	the suite mounts nothing real
6	Cost reported <i>no data</i> while 15 priced calls sat in the store — the session database engine was used to query the cost database	the suite’s cost tests inject the aggregate directly

9 Honest position

What is now known. The platform generates real code, records real costs, and refuses to claim success it did not verify. One session costs approximately **\$0.085** on the original (broken-parser) run, and **\$0.025** once the parser is fixed, because a stage that passes first time costs one request instead of three. **The agent does satisfy its own house standard on the first attempt at every stage** — the opposite of the claim originally made here.

CORRECTED. The sentence that stood here — “The agent does not satisfy its own house standard on the first attempt at any stage, and one stage in five did not converge” — described a parsing failure, not a compliance failure.

What is not known. Whether the generated artifacts were any good — because they were discarded before they could be diagnosed. The conformance measure’s real-world variance is therefore still unmeasured, and that measurement is the precondition for the optimization work.

What should not be concluded. That the agent is unreliable in the sense of being incapable. Fifteen rejections are rejections of *compliance*, not demonstrations of inability. The distinction is the whole reason procedural optimization is worth attempting here — and the whole reason it must be measured rather than assumed.